

## Lucrarea de laborator nr. 7

# Control comportamental cu automate finite

### Scopul lucrării

Scopul lucrării este familiarizarea cu principiile de proiectare și implementare a automatelor finite (FSM – Finite State Machine) în sisteme electronice incorporate, prin realizarea unei aplicații modulare care controlează comportamentul unui sistem simplu (ex.: LED) în funcție de evenimente externe (ex.: apăsarea unui buton). Lucrarea urmărește dezvoltarea unui sistem structurat, cu separarea logicii FSM într-un modul dedicat, integrarea interfețelor hardware-software (buton, LED, STDIO/LCD) și utilizarea serviciilor FreeRTOS pentru temporizare și sincronizare. Se pune accent pe documentarea arhitecturii, a diagramelor de stare și pe testarea funcționalității în mediu real sau simulare.

### Obiectivele lucrării

- Familiarizarea cu principiile de funcționare ale automatelor finite (FSM) în sisteme electronice incorporate.
- Implementarea unui sistem modular pentru controlul comportamental, utilizând un automat finit cu două stări (e.g ON/OFF).
- Utilizarea interfeței STDIO (Serial sau LCD) pentru afișarea și raportarea stării curente a automatului.
- Separarea logicii FSM într-un modul software dedicat, pentru facilitarea testării și reutilizării.
- Proiectarea și implementarea unui sistem de control complex pe baza automatelor finite cu mai multe stări și cereri externe (buton/senzor).
- Integrarea serviciilor FreeRTOS (taskuri, semafoare, delayuri) pentru sincronizarea și temporizarea tranzițiilor de stare ale semaforului.
- Documentarea arhitecturii software, a diagramelor de stare și a interfețelor hardware-software utilizate.

## 7.1 Analiza domeniului de aplicare

Automat Finit este o abstracție ce descrie o soluție a unei probleme care definește comportamentul sistemului sub forma de mecanism care își schimbă stările ca reacție la intrările sistemului și produce ieșiri corespunzătoare.

Caracteristicile principale ale unui automat finit sunt reprezentate de următoarele elemente

- Număr finit de stări, una este definită ca inițială
- Număr finit de intrări în sistem
- Număr finit de ieșiri generate
- Funcție de transfer pentru tranzițiile dintre stări
- Funcție de definire a ieșirilor

Totodata acesta poate un Automat finit poate fi definit ca si o "mașină cu un număr finit de stări", un model de comportament compus din stări, tranziții și acțiuni unde:

- Stările - stochează informații despre trecut, adică reflectă schimbările intrării de la inițializarea sistemului până în momentul de față.
- Tranzițiile - indică o schimbare de stare și este descrisă de o condiție care este nevoie să fie îndeplinită pentru a declanșa tranziția.
- Acțiunile - sunt descrieri a unor activități ce urmează a fi executată la un anumit moment.

Un exemplu de definire a unui automat finit cu evidentierea componentelor mentionate pentru controlul unei usi este prezentat in figura 7.1.

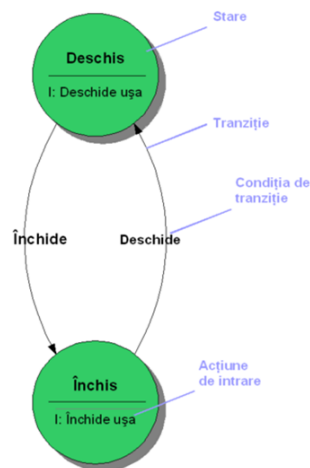


Figure 7.1: Exemplu de diagramă de stări pentru un automat finit de control al unei uși. [28]

Din punct de vedere al automatelor finite vom identifica doua tipuri: Automat Moore si Automat Mealy. in esenta, diferenta dintre aceste doua tiouri de automate este data de modul in care sunt generate iesirile.

**Automatul Moore** va avea functia de tranzitie a satarii dependenta de starea curenta, si intrarile sistemului, iar iesire depind doar de starea curenta a automatului:

- $NextState = f(Input, CurrentState)$
- $Output = g(CurrentState)$

Diagrama structurala a automatului Moore va arata ca si in figura 7.2.

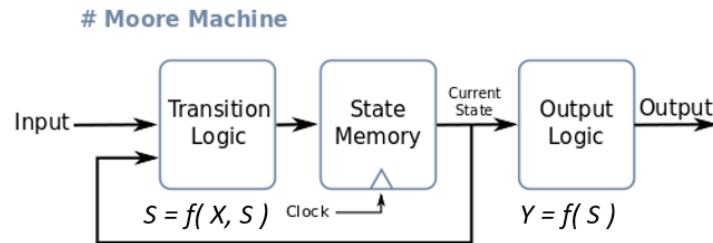


Figure 7.2: Diagrama structurală a unui automat finit Moore. [7]

Iar diagram de stări a unui automat finit Moore este prezentată în figura 7.3.

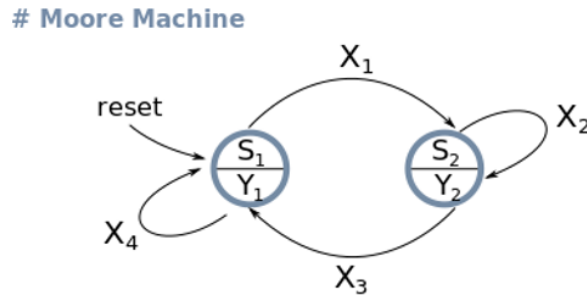


Figure 7.3: Diagrama de stări a unui automat finit Moore. [7]

Analizând o stare a automatului Moore, vom observa că ieșirea este definită doar de starea curentă, iar în diagrama de stări chiar în interiorul elementului de stare, ca în figura 7.4.

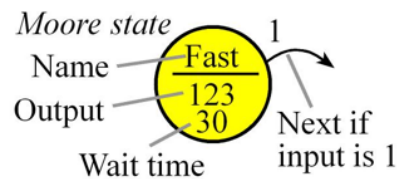


Figure 7.4: Exemplu de stare a unui automat finit Moore. [25]

Unde elementele unei stări Moore sunt:

- **Numele stării** (ex: Fast) — identifică starea curentă.
- **Ieșirea asociată** (ex: 123) — valoarea generată în starea curentă.
- **Timpul de așteptare** (ex: 30) — durata de menținere a stării.
- **Tranziția către starea următoare** — determinată de intrare (ex: input=1).

Astfel, într-un automat Moore, ieșirea depinde doar de starea curentă, iar tranzițiile sunt controlate de valorile de intrare. Diagrama ilustrează: fiecare stare este un cerc cu nume, ieșire și timp, iar săgețile indică tranzițiile posibile în funcție de intrare.

Evaluarea automatului Moore poate fi structurată profesional în pseudocod, conform listingului 7.1:

Listing 7.1: Structura generală a unui automat finit Moore

```

1  /*
2   * Automat finit Moore - flux principal
3   */
4  void TaskMooreFSM(void) {
5      // 1. Evaluatează ieșirile, dependente doar de starea curentă
6      Output = MooreOutput(CurrentState);
7  }

```

```

8 // 2. Menține perioada specifică stării curente (ex: delay)
9 Wait(StateDelay[CurrentState]);
10
11 // 3. Colectează intrările externe (ex: buton, senzor)
12 Input = ReadInput();
13
14 // 4. Evaluează starea următoare pe baza stării curente și a intrării
15 CurrentState = MooreNextState(CurrentState, Input);
16 }

```

**Automatul Mealy** , la randul sau, va avea functia de tranzitie a satarii dependenta de starea curenta, si intrarile sistemului, iar iesirile vor depinde atat de starea curenta cat si de intrarile sistemului:

- $NextState = f(Input, CurrentState)$
- $Output = h(Input, CurrentState)$

Diagrama structurala a automatului Mealy va arata ca si in figura 7.5.

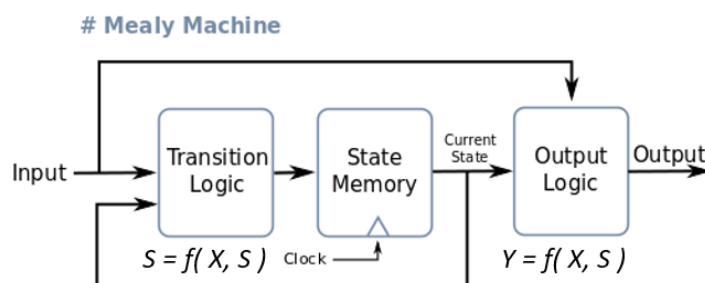


Figure 7.5: Diagrama structurala a unui automat finit Mealy. [7]

Iar diagram de stari a unui automat finit Mealy este prezentata in figura 7.6.

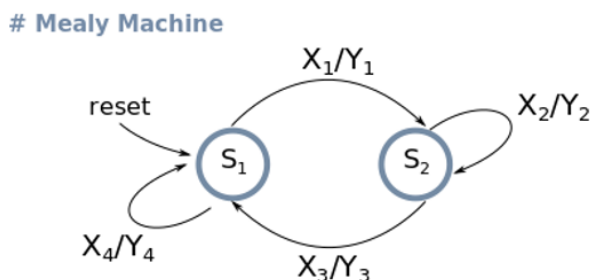


Figure 7.6: Diagrama de stări a unui automat finit Mealy. [7]

Analizand o starea a automatului Mealy, vom observa ca iesirea este definita atat de starea curenta cat si de intrarile sistemului, iar in diagrama de stari chiar in interiorul elementului de stare, ca in figura 7.7.

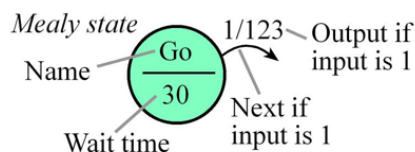


Figure 7.7: Exemplu de stare a unui automat finit Mealy. [25]

Unde elementele unei stări Mealy sunt:

- **Numele stării** (ex: Go) — identifică starea curentă.

- **Ieșirea asociată** (ex: 1/123) — valoarea generată, dependentă de starea curentă și de intrare (ex. dacă input=1).
- **Timpul de așteptare** (ex: 30) — durata de menținere a stării.
- **Tranziția către starea următoare** — determinată de intrare (ex: input=1).

Evaluarea automatului Mealy poate fi structurată în pseudocod, conform listingului 7.2:

Listing 7.2: Structura generală a unui automat finit Mealy

```

1  /*
2   * Automat finit Mealy - flux principal
3   */
4  void TaskMealyFSM(void) {
5      // 1. Colectează intrările externe (ex: buton, senzor)
6      Input = ReadInput();
7
8      // 2. Evaluează ieșirea, dependentă de starea curentă și de intrare
9      Output = MealyOutput(CurrentState, Input);
10
11     // 3. Menține perioada specifică stării curente (ex: delay)
12     Wait(StateDelay[CurrentState]);
13
14     // 4. Evaluează starea următoare pe baza stării curente și a intrării
15     CurrentState = MealyNextState(CurrentState, Input);
16 }

```

**Mealy vs Moore** ca și automate reprezintă în esență aceeași funcționalitate, dar cu diferențe în implementare și comportament. Principalele diferențe dintre cele două tipuri de automate sunt:

- **Dependența ieșirilor:** În automatul Moore, ieșirile depind doar de starea curentă, în timp ce în automatul Mealy, ieșirile depind atât de starea curentă cât și de intrări.
- **Numarul de stări:** Automatul Mealy poate avea un număr mai mic de stări decât automatul Moore pentru aceeași funcționalitate, deoarece ieșirile pot varia în funcție de intrări.
- **Timpul de răspuns:** Automatul Mealy poate avea un timp de răspuns mai rapid la schimbarea intrărilor, deoarece ieșirile se pot schimba imediat ce intrările se schimbă, fără a aștepta o tranziție de stare.
- **Complexitatea proiectării:** Automatul Moore este adesea considerat mai simplu de proiectat și înțeles, deoarece ieșirile sunt determinate doar de stări, în timp ce automatul Mealy poate fi mai complex din cauza dependenței ieșirilor de intrări.
- **Stabilitatea ieșirilor:** Ieșirile automatului Moore sunt mai stabile, deoarece nu se schimbă decât la tranziții de stare, în timp ce ieșirile automatului Mealy pot fluctua mai des în funcție de intrări.
- **Utilizarea în aplicații:** Automatul Moore este adesea utilizat în aplicații unde stabilitatea ieșirilor este critică, în timp ce automatul Mealy este preferat în aplicații unde timpul de răspuns rapid este esențial.
- **Implementarea hardware:** Automatul Mealy poate necesita mai puține resurse hardware în anumite cazuri, deoarece poate avea un număr mai mic de stări.
- **Diagrame de stare:** În diagramele de stare, ieșirile automatului Moore sunt reprezentate în interiorul stării, în timp ce ieșirile automatului Mealy sunt asociate cu tranzițiile dintre stări.
- **Exemplu de aplicație:** Un exemplu clasic de automat Moore este un semafor, unde culorile luminii depind doar de starea curentă. Un exemplu de automat Mealy este un sistem de control al vitezei unui motor, unde ieșirea (viteza motorului) depinde

atat de starea curenta (modul de operare) cat si de intrari (comenzi de accelerare sau franare).

**Implementare unui Automat Finit** poate fi realizata printr-o metoda eleganta prezentata EDX: University of Texas at Austin cursul Embedded Systems - Shape the World: Designing a FSM [25] unde realizarea automatului finit este structurata in acei pasi prezintati pentru abele Automate, iar evaluarea urmatoarei stari se va reduce la selectare din reprezentarea tabelara a automatului.

In asa mod pentru o aplicatia Button-LED, unde un buton controleaza starea unui LED (ON/OFF), automatul finit poate fi definit ca un automat Moore cu doua stari, reprezentat de figura 7.8.

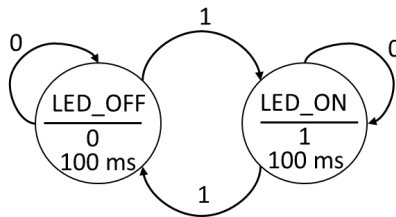


Figure 7.8: Diagrama de stări a automatului finit Button-LED.

iar reprezentarea tabelara a automatului fiind ca in tabelul 7.1 unde fiecare stare este definita prin un rand al tabelului continand descrierea starii curente prin retinerea in stare, valoarea generata la iesire, si tranzitiile dependente de intrare.

| Num | Name    | Out | Delay  | In = 0  | In = 1  |
|-----|---------|-----|--------|---------|---------|
| 0   | LED_OFF | 0   | 100 ms | LED_OFF | LED_ON  |
| 1   | LED_ON  | 1   | 100 ms | LED_ON  | LED_OFF |

Table 7.1: Tabel de stări pentru automatul Button-LED

Pentru implementare in cod C, se va defini o structura de definire a unei stari, care va sta la baza definirii automatului finit, care va reflecta unu la unu reprezentarea tabelara a automatului finit. aceasta definire a automatului pentru aplicatia Button-Led se va prezenta ca in listingul 7.3.

Listing 7.3: Exemplu de automat finit Button-LED în C

```

1 #define LED_OFF_STATE 0
2 #define LED_ON_STATE 1
3
4 struct State {
5     unsigned long Out;          // Led State
6     unsigned long Time;        // delay in 10ms units
7     unsigned long Next[2];     // next state for inputs 0,1
8 };
9 typedef const struct State STyp;
10
11 STyp FSM[2]={
12     {0,10,{LED_OFF_STATE, LED_ON_STATE }},
13     {1,10,{LED_ON_STATE, LED_OFF_STATE }}
14 };
  
```

Pentru a pune in functiune atomatul de contro al aplicatiei Button-LED va fi necesara initializarea periferiilor, cat si stabilirea starii initiale a automatului, aici - LED OFF.

Listing 7.4: Inițializare hardware și stare automat finit Button-LED

```

1 int FSM_State = LED_OFF_STATE;
2
3 void setup()
  
```

```
4 {  
5     // Init Button  
6     pinMode(BUTTON_PIN, INPUT);  
7     // Init LED  
8     pinMode(LED_PIN, OUTPUT);  
9     // Init Initial State  
10    FSM_State = LED_OFF_STATE;  
11 }
```

Funcția principală `loop()` va implementa logica de evaluare a automatului finit prezentată mai sus respectând acei 4 pași, dar și medelul automatului Moore, unde ieșirea depinde doar de starea curentă. Astfel implementarea funcției `loop()` va arăta ca în listingul 7.5.

Listing 7.5: Funcția principală `loop()` pentru automatul finit Button-LED

```
1 void loop()  
2 {  
3     // 1. Output Based on current state  
4     int output = FSM[FSM_State].Out;  
5     digitalWrite(LED_PIN, output);  
6  
7     // 2. wait for time relevant to state  
8     delay(FSM[FSM_State].Time * 10);  
9  
10    // 3. Get Input  
11    int input = digitalRead(BUTTON_PIN);  
12  
13    // 4. Change state based on input and current state  
14    FSM_State = FSM[FSM_State].Next[input];  
15 }
```

## 7.2 Sarcina de laborator

### 7.2.1 Partea 1: Control comportamental cu Automate Finite – Button-LED

Se cere realizarea unei aplicații embedded care implementează un automat finit pentru controlul unui LED, în funcție de apăsarea unui buton. Sistemul va avea două stări: LED-ul stins (OFF) și LED-ul aprins (ON). Tranziția între aceste stări se va efectua la fiecare apăsare validă a butonului, cu implementarea logicii de debounce pentru eliminarea fluctuațiilor nedorite. Starea curentă a LED-ului va fi afișată în timp real prin interfața serială (`printf`) sau pe un LCD (2x16 sau 4x20), utilizând biblioteca `STDIO` sau `LCD`.

### 7.2.2 Partea 2: Control comportamental cu Automate Finite – Semafor Inteligent

Să se proiecteze și să se implementeze o aplicație embedded pentru controlul unui semafor inteligent la o intersecție în cruce cu două direcții de trafic: Est-Vest și Nord-Sud. Semaforul trebuie să comute prioritatea între direcții pe baza unei cereri active, simulată prin apăsarea unui buton sau activarea unui senzor digital. În lipsa cererii pentru direcția Nord, prioritatea va fi acordată direcției Est, care va rămâne verde. Tranzițiile de semnalizare vor respecta secvența rutieră, trecând de la verde la galben și apoi la roșu. Durata fiecărei stări (verde, galben, roșu) va fi gestionată cu delay-uri folosind funcțiile `FreeRTOS`, precum `vTaskDelay` sau `vTaskDelayUntil`. Controlul semaforului va fi realizat printr-un automat finit (FSM) pentru fiecare direcție. Cererea de traversare va fi generată prin apăsarea unui buton sau activarea unui senzor de proximitate, iar starea curentă a semaforului va fi afișată pe interfața `STDIO`, fie pe `Serial`, fie pe `LCD`.

**Cerințe suplimentare:** Pentru lucrările cu automate finite (FSM) în sisteme electronice incorporate, respectați următoarele recomandări pentru structură și validare:

- Structurați aplicația modular, separând logica FSM într-un modul software dedicat.
- Utilizați diagrame de stare pentru a vizualiza stările și tranzițiile automatului.
- Implementați debouncing software pentru butoane sau senzori digitali.
- Dacă folosiți FreeRTOS, creați taskuri dedicate pentru fiecare componentă majoră (FSM, buton, LED, afișare).
- Pentru comunicarea între taskuri, utilizați semafoare binare sau variabile globale protejate (mutex).
- Reutilizați modulele pentru butoane, LED-uri și interfață de afișare.
- Testați fiecare tranziție de stare conform scenariilor de validare definite.
- Documentați clar arhitectura, diagramele de stare, interfețele HW/SW și scenariile de testare.
- Consultați resursele teoretice despre FSM și proiectare modulară.
- Sistemul trebuie să răspundă la schimbarea semnalului cu o latență sub 100ms.
- Afișați starea curentă și evenimentele relevante către STDIO prin interfața serială sau LCD.
- utilizați instrumente de analiză și vizualizare a datelor (ex.: Arduino Serial Plotter) pentru a monitoriza performanța sistemului în timp real.

**Materiale si resurse** necesare pentru realizarea lucrării:

- *Componente hardware:* microcontroler compatibil (ESP32, STM32, Arduino), LED-uri roșu, galben, verde (semafor), buton cu rezistență pull-up/pull-down, rezistor 220  $\Omega$  pentru LED, breadboard, cabluri de conexiune, sursă de alimentare USB/baterie, senzori digitali pentru cerere Nord (opțional).
- *Resurse software:* Visual Studio Code cu PlatformIO, biblioteci STDIO (Serial), LiquidCrystal\_I2C, FreeRTOS, simulator hardware (Proteus, Wokwi), emulator terminal serial (Monitor Serial PlatformIO, TeraTerm, Putty), instrument de diagrame (draw.io, Lucidchart).

**NOTA:** Realizarea sarcinii se va desfășura în două etape distincte, definite ca doua probleme în cadrul lucrării de laborator. aceste probleme vor fi realizate conform metodologiei prezentate în anexa de problema rezolvată. Fiecare etapă va fi documentată conform cerințelor metodologice, incluzând cerințe, schite arhitecturale, diagramele de flux, scenariile de testare și rezultatele obținute. Fiecare parte a lucrării, problema este planificată pentru o sedință de 2 ore academice.

### 7.3 Mersul Lucrării

1. Sa se analizeze si sa se inteleaga Sarcina lucrării de laborator, si anume Analiza situației în domeniu si Contextul tehnologic al aplicației dezvoltate conform secțiunii 1.1.
2. sa se familiarizeze cu problema propusa spre elaborare, si sa se identifice obiectivele si sarcina individuala vezi A.1.1.
3. Sa se analizeze componentele si resursele utilizate in realizarea aplicatiei din cadrul lucrării de laborator.
4. In urma analizei sarcinii lucrării si definitiei problemei propuse, sa se defineasca un set de cerințe tehnice acoperind aspectele functionale si nefunctionale ale problemei,



vezi A.1.2

5. În urma analizei cerințelor tehnice elaborate să se definească scenarii de testare și criteriilor de validare cu minim 1 test per cerința care să răspundă la întrebarea *cum mă asigur că cerința se satisface*, vezi A.1.3
6. să se identifice componentele ce compun sistemul aplicației, să se organizeze într-o listă cu descrierea rolului acestora în aplicație.
7. Să se realizeze diagrama structurală cu reprezentarea componentelor și interacțiunii între acestea așa cum este prezentat în secțiunea A.1.4
8. Să se realizeze arhitectura pe nivele pentru interfața hardware-software pentru fiecare dintre interfețe vezi A.1.4.
9. Să se realizeze diagramele de comportament pentru funcționalitățile Componentelor cu diagrame de flux stări sau de secvență, după caz, ca în A.1.5
10. să se elaboreze schița electrică a aplicației într-un mediu de simulare, vezi A.1.6.
11. să se creeze structura de directoare și fișiere pentru implementarea aplicației care va reflecta arhitectura definită de sistem, și de interfața hardware Software vezi A.1.7.
12. Să se implementeze driverele de interfața hardware-software pentru fiecare componentă electronică utilizată în aplicație acoperind nivelele ECAL și SRV după caz A.1.7.
13. Să se implementeze comportamentul aplicației urmând diagrama de flux definită, A.1.7.
14. Să se simuleze funcționarea aplicației în medii precum Wokwi sau PlatformIO. cu o eventuală rulare pe hardware real.
15. Să se testeze și valideze aplicația conform scenariilor de testare și criteriilor de validare definite anterior, vezi exemplu în A.1.8.
16. să se documenteze rezultatele testării cu observații și recomandări pentru îmbunătățiri, A.1.8.
17. Să se optimizeze aplicația pe baza rezultatelor testării și validării
18. să se documenteze toate etapele realizate în cadrul lucrării de laborator conform cerințelor pentru raportul lucrării de laborator expuse în 1.1.5.

## 7.4 Întrebări de control

1. Ce este un automat finit și care sunt elementele sale (stări, tranziții, evenimente)?
2. Cum se implementează un FSM în limbaj C/C++ pentru microcontrolere?
3. Ce este debouncing-ul și de ce este necesar în gestionarea unui buton?
4. Cum se poate monitoriza starea automatului în timp real prin interfața serială sau LCD?
5. Care sunt avantajele separării logicii FSM într-un modul dedicat?
6. Ce beneficii aduce extinderea FSM-ului cu o stare suplimentară în funcționalitatea aplicației?
7. Ce este un automat finit și cum se aplică în controlul semafoarelor?
8. Cum se utilizează FreeRTOS pentru a sincroniza taskuri și a controla temporizările?
9. Ce rol are semaforul binar în comunicarea între taskuri?
10. Cum se gestionează cererile multiple în cadrul automatului finit?
11. Care sunt avantajele controlului bazat pe cerere în intersecțiile inteligente?
12. Ce îmbunătățiri pot fi aduse sistemului pentru creșterea siguranței și eficienței?

## 7.5 Grila de evaluare a lucrării de laborator

1. **Analiza sarcinii și contextului tehnologic** (12p)
  - (a) Analiza situației în domeniu și contextul tehnologic (1.1) (4p)
  - (b) Identificarea resurselor externe și referințe în bibliografie (4p)
  - (c) Analiza componentelor și resurselor utilizate (4p)
2. **Definirea cerințelor tehnice și scenariilor de testare** (13p)
  - (a) Identificarea obiectivelor și sarcinii individuale (A.1.1) (3p)
  - (b) Formularea cerințelor funcționale și non-funcționale (A.1.2) (5p)
  - (c) Definirea scenariilor de testare și criteriilor de validare (A.1.3) (5p)
3. **Proiectarea arhitecturală și modelare comportamentală** (18p)
  - (a) Listarea componentelor și descrierea rolului fiecăreia (3p)
  - (b) Diagrama structurală cu componente și interacțiuni (A.1.4) (4p)
  - (c) Arhitectura pe nivele pentru interfața hardware-software (4p)
  - (d) Diagrame comportament pentru funcționalități (flux, stări, secvență) (A.1.5) (7p)
4. **Proiectarea și simularea schiței electrice** (13p)
  - (a) Elaborarea schiței electrice în mediu de simulare (A.1.6) (4p)
  - (b) Reprezentarea corectă a conexiunilor și dimensionarea componentelor (2p)
  - (c) Simularea funcționării aplicației în medii precum Wokwi/PlatformIO (3p)
  - (d) Rularea aplicației pe HW real (10p)
5. **Implementarea software modulară** (18p)
  - (a) Structura de directoare și fișiere conform arhitecturii (A.1.7) (4p)
  - (b) Implementarea de interfață HW-SW conform proiectării în detaliu (6p)
  - (c) Implementarea comportamentului aplicației conform proiectării în detaliu (6p)
  - (d) Comentarii explicative și documentare în cod (2p)
6. **Simulare, testare și validare** (16p)
  - (a) Testarea și validarea conform scenariilor definite (A.1.8) (8p)
  - (b) Documentarea rezultatelor testării, observații și recomandări (5p)
  - (c) Optimizarea aplicației pe baza rezultatelor testării și validării (3p)
7. **Documentare finală** (10p)
  - (a) Documentarea etapelor realizate conform cerințelor raportului (1.1.5) (5p)
  - (b) Observații, recomandări, concluzii generale (5p)

**Total: 100p**